Informe final de auditoría de contratos inteligentes

HackChain - Token HACK y mecanismos del ecosistema

Julian Ruiz - Blockchain Engineer / Smart Contract Security Reviewer

11 de julio de 2026

Índice

1 Dictamen ejecutivo **2**
1.1 Resumen de conformidad de los 30 mecanismos . . . 3
1.2 Riesgos de mayor prioridad . . . 3
1.3 Criterios de ingeniería revisados . . . 3

2 Alcance y fuentes **4**
2.1 Código auditado . . . 4
2.2 Documentación de negocio incorporada . . . 4
2.3 Fuera de alcance . . . 5

3 Metodología y evidencia **5**

4 Arquitectura y modelo de confianza **5**
4.1 Activos protegidos . . . 5
4.2 Actores privilegiados . . . 5
4.3 Invariantes mínimos requeridos . . . 6

5 Matriz de conformidad mecanismo por mecanismo **6**

6 Índice de hallazgos técnicos **9**

7 Hallazgos High **10**
7.1 H-01 - Referidos cobrables inmediatamente y explotables mediante Sybil . . . 10
7.2 H-02 - El principal de staking queda condicionado al pago de recompensas . . . 11
7.3 H-03 - Vistas académicas ilimitadas permiten apropiarse del pool de educadores . . . 12
7.4 H-04 - Las penalizaciones dependen del allowance voluntario del infractor . . . 12
7.5 H-05 - La base de penalización usa saldo movible y la venta masiva se evalúa post-venta . . . 13

8 Hallazgos Medium **14**
8.1 M-01 - `deposit()` infla contabilidad sin transferir tokens . . . 14
8.2 M-02 - Exención de comisión inmediata y elegibilidad obsoleta . . . 14
8.3 M-03 - Reclamación de reputación falla al cruzar la época . . . 14
8.4 M-04 - Eventos off-chain sin IDs únicos ni idempotencia . . . 15
8.5 M-05 - `Ownable` y `AccessControl` se transfieren de manera divergente . . . 15
8.6 M-06 - Bonos de recruiter no prueban actividad, registro ni KYC coherente . . . 15

9 Hallazgos Low e Informational **16**
9.1 L-01 - Transferencias ERC-20 no seguras . . . 16
9.2 L-02 - Índice de stake validado tarde . . . 16
9.3 L-03 - Membresía avanzada vencida conserva estado activo . . . 16
9.4 L-04 - Bucles y lotes no acotados . . . 16
9.5 L-05 - Patrocinador sobrescribible y sin reembolso . . . 16
9.6 I-01 - Épocas de 30 días no equivalen a meses calendario . . . 16
9.7 I-02 - Recompensas no reservadas . . . 16

9.8 I-03 - Despliegue y roles no reproducibles desde el repositorio recibido . . . 16
9.9 I-04 - Calidad y observabilidad . . . 16
9.10 L-06 - La pausa de emergencia no es granular y puede bloquear retiros . . . 17
9.11 I-05 - Mappings públicos y getters duplicados . . . 17
9.12 I-06 - NatSpec incompleto y parcialmente desalineado . . . 18
9.13 I-07 - Modernización condicional a `ReentrancyGuardTransient` . . . 18
9.14 Criterio revisado sin hallazgo - Convención `Contract__Error()` . . . 19

10 Brechas de conformidad de negocio **19**
10.1 BL-01 - Distribución, vesting y bloqueo de liquidez no se fuerzan on-chain (High) . . . 19
10.2 BL-02 - Comisión del 5% por transacción ausente (High) . . . 20
10.3 BL-03 - Clases personalizadas no implementadas (Medium) . . . 20
10.4 BL-04 - Membresía avanzada incompleta (Medium) . . . 20
10.5 BL-05 - Cancelación de membresía académica no implementada y fuentes contradictorias (Medium) . . . 20
10.6 BL-06 - Financiación OSS no valida roles, repositorio ni talentos (Medium) . . . 21
10.7 BL-07 - Suplantación y plagio no siguen el flujo documentado (Medium) . . . 21
10.8 BL-08 - Hechos de negocio dependen de confianza off-chain sin attestations (Medium) . . . 21
10.9 BL-09 - DAO/gobernanza documentada pero fuera del repositorio (Informational) . . . 21
10.10 BL-10 - Contradicciones documentales y exposición regulatoria (Medium) . . . 21

11 Arquitectura de remediación recomendada **22**
11.1 Capa 1 - Token y tesorería . . . 22
11.2 Capa 2 - Evidencia de negocio . . . 22
11.3 Capa 3 - Pagos y recompensas . . . 22
11.4 Capa 4 - Penalizaciones . . . 22

12 Plan de corrección priorizado **22**
12.1 Bloqueante antes de testnet pública . . . 22
12.2 Antes de mainnet . . . 22

13 Pruebas de aceptación mínimas después de corregir **22**

14 Guía de reproducción del paquete **23**

15 Limitaciones **23**

16 Conclusión final **23**

17 Apéndice A - Evidencia y entorno técnico **24**

18 Apéndice B - Hashes de documentos de negocio **24**

19 Apéndice C - Referencias técnicas **24**

20 Apéndice D - Nota sobre severidad **24**

1 Dictamen ejecutivo

Auditor: Julian Ruiz
Rol: Blockchain Engineer - Smart Contract Security Reviewer
LinkedIn: linkedin.com/in/julian-ruiz-v
Resultado: NO APTO PARA PRODUCCIÓN en el estado auditado.

El repositorio implementa una parte importante de los mecanismos propuestos, pero no garantiza de forma segura varias condiciones económicas centrales. Se confirmaron vulnerabilidades que permiten cobrar recompensas antes de cumplir los requisitos, fabricar actividad remunerada, bloquear la devolución del principal de staking y evitar penalizaciones. La revisión contra el modelo de negocio también identificó mecanismos ausentes, reglas documentadas que no están codificadas y contradicciones que deben resolverse antes de cerrar la especificación.

Severidad técnica	Cantidad
Critical	0
High	5
Medium	6
Low	6
Informational	7
Total	**24**

Además de los 24 hallazgos técnicos y observaciones de ingeniería, se registran **10 brechas de conformidad de negocio**. Estas brechas se presentan por separado para no duplicar severidades cuando una misma causa técnica afecta más de un mecanismo.

1.1 Resumen de conformidad de los 30 mecanismos

Estado	Cantidad	Interpretación
Cumple completamente	0	No se encontró un mecanismo que pueda considerarse completo, seguro y autosuficiente frente a toda la especificación.
Mayormente implementado	3	La función principal existe, pero todavía depende de validación privilegiada/off-chain o requiere endurecimiento.
Parcialmente implementado	25	Existe una aproximación, pero faltan condiciones, flujos, controles o distribución económica.
Ausente	2	No existe un contrato o flujo equivalente en el alcance auditado.

Los dos mecanismos ausentes son la **comisión del 5% por transacción en HACK** y el **pago/escrow de clases personalizadas con mínimo de 1.000 HACK**.

1.2 Riesgos de mayor prioridad

1. **Penalizaciones no ejecutables:** el infractor puede revocar o no otorgar allowance, mover sus tokens o depositarlos en staking para reducir la base de cálculo.
2. **Recompensas manipulables:** referidos se cobran inmediatamente y las vistas académicas pueden autogenerarse sin límite.
3. **Fondos de usuarios bloqueables:** la devolución del principal de staking depende de que el pool pueda pagar la recompensa.
4. **Tokenomics no forzados on-chain:** distribución, vesting, asignación de asesores y bloqueo de liquidez no están protegidos por contratos del alcance.
5. **Especificación inconsistente:** algunos diagramas, tablas, NatSpec y el plan de negocio definen resultados distintos para la misma acción.

1.3 Criterios de ingeniería revisados

Criterio revisado	Dictamen	Tratamiento en el informe
Errores con formato `Contract__Error()`	No obligatorio	Convención opcional. No se clasifica como vulnerabilidad ni se recomienda un cambio masivo solo por estilo.
`mapping public` junto con getters equivalentes	Sí aplica	Se añade I-05 por duplicación de ABI, bytecode y acoplamiento de la interfaz al almacenamiento.
Mejorar NatSpec	Sí aplica	Se añade I-06 por cobertura incompleta y comentarios que no siempre coinciden con el comportamiento.
Migrar a `ReentrancyGuardTransient`	Aplica de forma condicional	Se añade I-07. Es una modernización recomendada si la red y el toolchain soportan EIP-1153; no corrige por sí sola un exploit existente.
Parada de emergencia para evitar nuevos depósitos	Sí aplica	Se añade L-06. La pausa actual del token es demasiado global y también puede impedir retiros; se recomienda una pausa granular de entradas con salidas habilitadas.

Estos criterios no cambian los cinco riesgos High ni los seis Medium. Se clasifican como un hallazgo Low y tres observaciones Informational.

2 Alcance y fuentes

2.1 Código auditado

- Repositorio: Hackchain HACK Token Smart Contracts.
- Branch declarada: `main`.
- Commit congelado: `290377e9862aeabaa2814bcd0bac9ddad2707ce8`.
- Solidity: `0.8.24`.
- OpenZeppelin utilizado para reproducir el commit auditado: `5.0.2`.
- OpenZeppelin revisado para modernización: `5.6.1` (prueba de compatibilidad separada, sin modificar el código auditado).
- Contratos: 11.
- Líneas Solidity aproximadas: 3.052.
- Red objetivo documentada: Polygon.

2.2 Documentación de negocio incorporada

- **Mecanismos del token HACK**, 12 páginas: definición tabular de 30 mecanismos.
- **Plan de negocios**, 11 páginas: distribución, vesting, membresías, comisiones, proyecciones y rendimientos a inversionistas.
- **Whitepaper HackChain 1.1.0**, 19 páginas: arquitectura, propuesta de valor, tokenomics, gobernanza, roadmap y modelo de negocio.
- **Diagramas de mecanismos**, 1 lámina: decisiones y flujos de los 30 mecanismos.
- **Conoce Hack Chain**, 12 páginas: presentación comercial y resumen del ecosistema.

Los documentos se utilizaron como fuente de intención. Cuando dos fuentes discrepan, el informe no elige silenciosamente una: la contradicción se registra y se exige una decisión formal del equipo.

2.3 Fuera de alcance

- Backend, frontend, base de datos, WalletConnect, Pinata, OpenSea, Alchemy, KYC y servicios de identidad.
- Contratos de certificados ERC-721, identidades tokenizadas, DAO, vesting, venta o liquidez que no aparecieron en el repositorio recibido.
- Direcciones de despliegue, multisig, scripts de configuración de roles y políticas operativas reales.
- Revisión legal, fiscal o regulatoria. Las observaciones de documentación financiera requieren abogado especializado y no constituyen asesoría legal.

3 Metodología y evidencia

1. Congelación del commit y hashes SHA-256.
2. Revisión manual por contrato y revisión cruzada de flujos de tokens.
3. Modelado de roles, activos, estados de privilegio y dependencias externas.
4. Compilación exacta con `solc-js 0.8.24`.
5. Slither 0.11.5 y triage manual de 69 señales brutas.
6. Mocha, Ethers y Ganache: **16 pruebas de concepto ejecutadas satisfactoriamente**.
7. Foundry Forge 1.7.1: 7 pruebas passing, incluyendo fuzz de 256 ejecuciones.
8. Halmos 0.3.3 + Z3: contraejemplo simbólico del invariante de solvencia del pool.
9. Segunda verificación de los hallazgos frente a los documentos de negocio.
10. Revisión de convenciones de errores, getters, NatSpec, pausa de emergencia y compatibilidad de `ReentrancyGuardTransient` con OpenZeppelin 5.6.1, Solidity 0.8.24 y `evmVersion=cancun`.

Comando principal de reproducción:

```
npm ci
npm run compile
npm run test:js
```

Resultado observado:

```
Hackchain security proof-of-concepts: 16 passing
Hackchain business-logic conformance proof-of-concepts: 5 passing
Total: 16 passing (8s)
```

La advertencia de uWS de Ganache únicamente activa una implementación JavaScript más lenta; no altera el resultado funcional de las pruebas.

4 Arquitectura y modelo de confianza

4.1 Activos protegidos

- Oferta máxima y distribución de HACK.
- Principal depositado en staking.
- Balance real y contable del pool de incentivos.
- Pagos de membresías y porción destinada a tesorería/educadores.
- Recompensas por referidos, eventos, reputación, educación y contratación.
- Penalizaciones y bloqueo de perfiles.
- Fondos privados destinados a proyectos de código abierto.

4.2 Actores privilegiados

La mayoría de hechos de negocio se registra mediante `ADMIN_ROLE` o `ENFORCER_ROLE`. Por tanto, el sistema no es completamente trustless: la seguridad depende de que dichos roles no se comprometan y de que el backend no reenvíe eventos duplicados. El uso de un multisig reduce el riesgo de una clave individual, pero **no reemplaza** idempotencia, attestations firmadas ni límites económicos on-chain.

4.3 Invariantes mínimos requeridos

- Un usuario siempre puede recuperar su principal vencido, aunque no haya liquidez para recompensas.
- `poolBalance` nunca puede superar el balance ERC-20 real sin una deuda explícita y rastreable.
- Un hecho externo solo puede consumirse una vez.
- Las penalizaciones no pueden evitarse moviendo tokens después de la infracción.
- Una vista remunerable debe corresponder a un usuario, contenido y periodo únicos.
- La suma de tokens minteados/asignados por categoría debe respetar el cap y la distribución aprobada.

5 Matriz de conformidad mecanismo por mecanismo

#	Mecanismo documentado	Implementación observada	Estado	Brecha principal
1	Staking: 1.000/30 días -> 50; 10.000/365 días -> 1.000	`Staking`	Parcial	Principal y recompensa están acoplados; no existe reserva garantizada.
2	Referido: 1.000; máximo 5/mes; referido mantiene 1.000 por 1 mes	`Referral`	Parcial	Puede validarse inmediatamente y con wallets Sybil.
3	Membresía avanzada: 50k/140k/550k; 50/50; cancelación 1k	`Membership`	Parcial	Solo existe nivel mensual; no hay reembolso; estado vencido inconsistente.
4	Bono por grado avanzado: 50k y vinculación a identidad	`Talent`	Parcial	No se valida nivel ni identidad tokenizada; depende del enforcer.
5	Evento promocional >=10 asistentes: 30k al organizador	`Events`	Mayormente	`eventId` evita doble cobro, pero asistentes/organizador dependen del enforcer.
6	Sin comisión con 100k en staking durante >=12 meses	`Staking`	Parcial	Se activa inmediatamente y puede permanecer tras retirar el stake anual.
7	Suplantación: 10% y bloqueo/eliminación/corrección	`Penalties`	Parcial	No bloquea el perfil y la cobranza es evadible.
8	Comisión de 5% por transacción en HACK a tesorería	No existe	Ausente	No hay router, settlement ni consumo del estado `hasNoCommission`.

#	Mecanismo documentado	Implementación observada	Estado	Brecha principal
9	Venta masiva: >50% por holder >=1% de oferta	`Penalties`	Parcial	Usa balance posterior y supply entregado por el enforcer; definición del monto contradice documentos.
10	Talento asiste a 4 eventos en el mes: 4k	`Events`	Parcial	Sin IDs únicos; meses son épocas de 30 días.
11	Talento contratado: 5k, una vez/mes	`Talent`	Parcial	La contratación no tiene ID único ni prueba de registro.
12	Pago a educador por clase personalizada, mínimo 1k/h	No existe	Ausente	Falta escrow, confirmación, disputa y pago.
13	Membresía académica 30k/80k/330k; 50/50	`Membership`	Parcial	Precios y reparto existen; cancela-ción/reembolso del diagrama no existe.
14	No-show entrevista: 5%; permanencia mínima 15 min	`Penalties`	Parcial	No existen `interviewId`, prueba de 15 min ni snapshot; penalty evadible.
15	Mejor reputación Talento: 5k, claim días 1-3	`Reputation`	Parcial	El cambio de época puede invalidar una ventana vigente.
16	Mejor reputación Educador: 5k, claim días 1-3	`Reputation`	Parcial	Misma vulnerabilidad de época; votación/empate off-chain.
17	Mejor reputación Reclutador: 5k, claim días 1-3	`Reputation`	Parcial	Misma vulnerabilidad de época; votación/empate off-chain.
18	Primer evento académico finalizado y certificados emitidos: 5k	`Educator`	Parcial	La finalización y certificados no se prueban on-chain.
19	Cuatro eventos finalizados en el mismo mes: 4k	`Educator`	Parcial	Sin `eventId`, expuesto a replays y a época de 30 días.

#	Mecanismo documentado	Implementación observada	Estado	Brecha principal
20	Implementar API oficial: 50k	`Educator`	Parcial	Verificación completamente privilegiada y sin attesta- tion/versionado.
21	Tokenizar primeros 10 PDF para 10 talentos: 10k	`Educator`	Mayormente	Hay IDs/contadores, pero la validez del certificado y destinatario es off-chain.
22	Primeros 10 talentos activos formándose: 10k	`Educator`	Mayormente	Control de unicidad existe; actividad sigue siendo una afirmación del enforcer.
23	Financiar OSS GitHub por Edu- cador/Reclutador: 1k/10k/100k	`Talent`	Parcial	Cualquier wallet y `projectId=0` son aceptados; no hay refund ni verificación de roles.
24	Bono a educador cuando su talento es contratado: 5k	`Educator`	Parcial	Sin `hiringId`; selección estadística no está codificada.
25	Educador sin evento por 2 meses: 5% y bloqueo	`Penalties`	Parcial	Detección off-chain; saldo movible; allowance voluntario.
26	Plagio: 10%, bloqueo y pago al afectado	`Penalties`	Parcial	Flujo externo termina en tesorería, no automatiza pago al afectado; desbloqueo sin caso.
27	Reclutador activo 7 días: 50k; una identidad/wallet	`Recruiter`	Parcial	Solo mide tiempo; no actividad ni identidad única.
28	Contratar 4 talentos activos/mes: 40k	`Recruiter`	Parcial	Sin `hiringId` ni prueba de actividad.
29	KYC oficial: 200k	`Recruiter`	Parcial	No exige registro ni attestation KYC única.
30	Reclutador sin entrevistas por 2 meses: 5% y bloqueo	`Penalties`	Parcial	Detección off-chain; saldo/allowance permiten evasión.

6 Índice de hallazgos técnicos

ID	Severidad	Hallazgo	Contratos
H-01	High	Referidos cobrables inmediatamente y explotables mediante Sybil	ReferralSystem, StakingContract
H-02	High	El principal de staking queda condicionado al pago de recompensas	StakingContract, IncentivesPool
H-03	High	Vistas académicas ilimitadas permiten apropiarse del pool de educadores	MembershipSystem
H-04	High	Las penalizaciones dependen del allowance voluntario del infractor	PenaltySystem, HackToken
H-05	High	La base de penalización usa saldo movible y la venta masiva se evalúa post-venta	PenaltySystem, StakingContract
M-01	Medium	`deposit()` infla contabilidad sin transferir tokens	IncentivesPool
M-02	Medium	Exención de comisión inmediata y elegibilidad obsoleta	StakingContract
M-03	Medium	Reclamación de reputación falla al cruzar la época	ReputationBunises
M-04	Medium	Eventos off-chain sin IDs únicos ni idempotencia	Múltiples
M-05	Medium	`Ownable` y `AccessControl` se transfieren de manera divergente	HackToken
M-06	Medium	Bonos de recruiter no prueban actividad, registro ni KYC coherente	RecruiterBonuses
L-01	Low	Transferencias ERC-20 no usan `SafeERC20` de forma consistente	MembershipSystem, PenaltySystem
L-02	Low	El índice de stake se usa antes de validarse	StakingContract
L-03	Low	Membresía avanzada vencida conserva `active=true`	MembershipSystem
L-04	Low	Bucles/lotes no acotados pueden quedar fuera de gas	StakingContract, TalentBonuses
L-05	Low	Patrocinador sobrescribible y ausencia de reembolso	TalentBonuses

ID	Severidad	Hallazgo	Contratos
L-06	Low	La pausa de emergencia no es granular y puede bloquear retiros	HackToken, StakingContract, MembershipSystem, IncentivesPool
I-01	Info	“Mes” se modela como época global de 30 días	Múltiples
I-02	Info	Recompensas no reservadas y dependientes de liquidez futura	Múltiples
I-03	Info	Faltan scripts, CI, configuración reproducible de roles y despliegues	Repositorio
I-04	Info	Deuda técnica, estado muerto y observabilidad incompleta	Múltiples
I-05	Info	Mappings públicos y getters duplicados amplían ABI y acoplan almacenamiento	Múltiples
I-06	Info	NatSpec incompleto y parcialmente desalineado con la lógica	Múltiples
I-07	Info	Modernización condicional a `ReentrancyGuardTransient` y OpenZeppelin actual	Múltiples

7 Hallazgos High

7.1 H-01 - Referidos cobrables inmediatamente y explotables mediante Sybil

Requisito de negocio. El referido debe mantener al menos 1.000 HACK en staking durante un mes; el referrer recibe 1.000 HACK y tiene un máximo de cinco referidos al mes.

Causa. `validateReferral()` consulta el total depositado, pero no comprueba la antigüedad ni la duración concreta de una posición. Tampoco vincula el registro del referido con un stake determinado o una identidad única.

Flujo de reproducción local.

1. El atacante controla A (referrer) y B (referred).
2. B deposita 1.000 HACK por 30 días.
3. B registra a A como referrer.
4. Se valida el referido en el mismo bloque o inmediatamente después.
5. A cobra 1.000 HACK aunque no transcurrió el mes.
6. El atacante repite con A2, A3, etc. para distribuir el límite entre identidades Sybil.

Impacto. Inflación de recompensas y drenaje del pool sin adquisición/retención real del usuario referido.

Solución recomendada. Vincular cada referido a `stakeId`, registrar el momento de asociación antes de la elegibilidad, consumir un `referralId` único y exigir el vencimiento real. El control Sybil requiere identidad/KYC o un score de humanidad fuera de la cadena; una limitación por wallet no es suficiente.

```
mapping(bytes32 => bool) public referralConsumed;
```

```
function validateReferral(
    bytes32 referralId,
    address referrer,
    address referred,
    uint256 stakeId
) external onlyRole(VALIDATOR_ROLE) {
    require(!referralConsumed[referralId], "referral used");
    require(referrer != referred, "self referral");

    Stake memory s = staking.getStake(referred, stakeId);
    require(s.amount >= 1_000 ether, "insufficient stake");
    require(s.duration >= 30 days, "wrong duration");
    require(block.timestamp >= s.startTime + 30 days, "not matured");
    require(s.active, "stake inactive");

    referralConsumed[referralId] = true;
    incentivesPool.distribute(referrer, 1_000 ether, "referral");
}
```

Prueba: `HC-H01` y `test_ReferralRewardCanBeClaimedImmediatelyAfterStake`.

7.2 H-02 - El principal de staking queda condicionado al pago de recompensas

Causa. `unstake()` intenta devolver el principal y pagar la recompensa en la misma transacción. Si `IncentivesPool.distribute()` revierte por insolvencia, pausa o permisos, la EVM revierte también la devolución del principal.

Flujo de reproducción local.

1. Usuario deposita 1.000 HACK.
2. Se avanza el tiempo 30 días.
3. El pool de incentivos queda sin saldo o sin `DISTRIBUTOR_ROLE` para staking.
4. Usuario llama `unstake(stakeId)`.
5. El pago de recompensa revierte.
6. Toda la transacción revierte y el principal permanece bloqueado.

Impacto. Congelamiento potencial de todos los depósitos vencidos por una dependencia económica/operativa externa.

Solución recomendada. Separar principal y recompensa. La falta de fondos solo debe afectar el reward. Registrar deuda `pendingRewards`, permitir claim posterior y añadir retiro de emergencia del principal.

```
function unstake(uint256 stakeId) external nonReentrant {
    Stake storage s = _stake(msg.sender, stakeId);
    require(block.timestamp >= s.unlockAt, "locked");
    require(s.active, "inactive");

    s.active = false;
    uint256 principal = s.amount;
    uint256 reward = _rewardFor(s);

    hackToken.safeTransfer(msg.sender, principal); // nunca depende del pool
    pendingRewards[msg.sender] += reward;
    emit PrincipalWithdrawn(msg.sender, stakeId, principal);
}

function claimReward(uint256 amount) external nonReentrant {
    require(pendingRewards[msg.sender] >= amount, "exceeds debt");
    pendingRewards[msg.sender] -= amount;
    incentivesPool.distribute(msg.sender, amount, "staking_reward");
}
```

Prueba: `HC-H02` y `test_EmptyRewardPoolBlocksPrincipalWithdrawal`.

7.3 H-03 - Vistas académicas ilimitadas permiten apropiarse del pool de educadores

Causa. Todo miembro académico activo puede ejecutar `registerContentView(educator)` ilimitadamente. No existe `contentId`, `viewId`, viewer único, límite por periodo ni prohibición de auto-vistas.

Flujo de reproducción local.

1. El atacante obtiene `EDUCATOR_ROLE` y compra membresía académica.
2. Hay fondos en `pendingEducatorPool` provenientes de otras membresías.
3. Llama cientos de veces `registerContentView(attacker)`.
4. Sus vistas representan el 100% del contador.
5. Ejecuta `claimEducatorRewards()` y recibe el pool completo.

Impacto. Apropiación de ingresos destinados a educadores legítimos y destrucción del modelo de reparto por consumo.

Solución recomendada. Registrar vistas mediante attestation EIP-712 de un servicio autorizado, con `viewId`, usuario, educador, contenido, ciclo, vencimiento y nonce. Prohibir auto-vistas y cerrar el ciclo antes de distribuir.

```
mapping(bytes32 => bool) public usedView;

function registerContentView(View calldata v, bytes calldata sig) external {
    require(v.viewer != v.educator, "self view");
    require(v.expiresAt >= block.timestamp, "expired");
    bytes32 viewId = keccak256(
        abi.encode(v.viewer, v.educator, v.contentId, v.cycleId)
    );
    require(!usedView[viewId], "duplicate view");
    require(_verifyViewSigner(v, sig), "invalid attestation");

    usedView[viewId] = true;
    views[v.cycleId][v.educator] += 1;
    totalViews[v.cycleId] += 1;
}
```

Prueba: `HC-H04` y `test_EducatorCanSelfGenerateViewsAndDrainEducatorPool`.

7.4 H-04 - Las penalizaciones dependen del allowance voluntario del infractor

Causa. `_applyPenalty()` cobra mediante `transferFrom(user, destination, amount)`. El infractor puede no aprobar o revocar la aprobación. Como el bloqueo se registra después, también se revierte.

Flujo de reproducción local.

1. La wallet sancionable mantiene HACK, pero no aprueba `PenaltySystem`.
2. El enforcer llama una función de penalización.
3. `transferFrom()` revierte por allowance insuficiente.
4. No se transfieren tokens y `profileBlocked` no cambia.

Impacto. Los mecanismos disciplinarios son opcionales para el propio infractor.

Soluciones posibles.

- **Modelo de deuda:** bloquear primero el perfil, registrar deuda y exigir pago para desbloquear. Es el modelo menos invasivo.
- **Escrow/collateral:** requerir un saldo depositado antes de usar servicios sujetos a penalización.
- **Hook limitado en el token:** permitir a un contrato de sanciones auditable mover/quemas saldos, con límites, multisig y timelock. Es más coercitivo y aumenta el riesgo de centralización.

```
function imposePenalty(address user, bytes32 caseId, uint256 amount)
    external onlyRole(ENFORCER_ROLE)
{
    require(!cases[caseId].exists, "case used");
    cases[caseId] = Case({user: user, debt: amount, exists: true, paid: false});
    profileBlocked[user] = true; // no depende de transferFrom
    emit PenaltyDebtCreated(caseId, user, amount);
}

function payPenalty(bytes32 caseId) external nonReentrant {
    Case storage c = cases[caseId];
    require(msg.sender == c.user && !c.paid, "invalid case");
    hackToken.safeTransferFrom(msg.sender, incentivesPool, c.debt);
    c.paid = true;
    profileBlocked[msg.sender] = false;
}
```

Prueba: `HC-H05` y `test_PenaltyCanBeAvoidedByWithholdingAllowance`.

7.5 H-05 - La base de penalización usa saldo movible y la venta masiva se evalúa post-venta

Causa. Las sanciones porcentuales leen `balanceOf(user)` al momento de ejecución. El usuario puede mover o stakear su saldo antes de que el enforcer actúe. Para venta masiva, el contrato compara `saleAmount` y el umbral del 1% contra el **balance posterior a la venta**, y acepta `circulatingSupply` proporcionado por el enforcer.

Flujo A: base cero mediante staking.

1. Usuario posee 100.000 HACK.
2. Antes de la sanción deposita los 100.000 HACK en `StakingContract`.
3. `PenaltySystem.balanceOf(user)` devuelve cero.
4. El cálculo produce penalización cero y revierte `AmountMustBeGreaterThanZero`.

Flujo B: venta masiva no detectable.

1. Usuario poseía 2% de la oferta circulante.
2. Vende suficiente HACK para quedar por debajo de 1%.
3. El enforcer llama `applyMassSalePenalty` después de la venta.
4. El contrato evalúa el nuevo balance y concluye que no poseía 1%.
5. La sanción no puede aplicarse aunque sí cumplía el criterio antes de vender.

Impacto. Evasión general de sanciones y falsos negativos en el control anti-whale.

Solución recomendada. Definir un evento de infracción con snapshot verificable: `preBalance`, `saleAmount`, `circulatingSupply`, tx hash y bloque. Validarlo con attestation firmada o integrar la tasa en el propio flujo de venta/settlement. Para sanciones de plataforma, usar deuda calculada sobre un valor snapshot y no sobre el balance presente.

```
struct MassSaleAttestation {
    address user;
    uint256 preBalance;
    uint256 saleAmount;
    uint256 circulatingSupply;
    uint256 observedBlock;
    bytes32 txHash;
    uint256 nonce;
    uint256 deadline;
}

function registerMassSale(MassSaleAttestation calldata a, bytes calldata sig) external {
    require(_verifyOracle(a, sig), "invalid proof");
```

```
    require(a.preBalance * 100 >= a.circulatingSupply, "below 1 percent");
    require(a.saleAmount * 100 > a.preBalance * 50, "below 50 percent");
    uint256 debt = a.saleAmount * 5 / 100; // elegir esta base formalmente
    _createDebt(a.user, a.txHash, debt);
}
```

Pruebas: BL-P02 y BL-P03.

8 Hallazgos Medium

8.1 M-01 - deposit() infla contabilidad sin transferir tokens

IncentivesPool.deposit(amount, reason) incrementa poolBalance y totalReceived, pero no ejecuta una transferencia ni verifica el balance real. Un DEPOSITOR_ROLE puede crear contabilidad ficticia; las distribuciones fallarán más tarde.

Reproducción: otorgar el rol a una wallet sin tokens, llamar deposit(1_000 ether, ...) y comparar poolBalance() con actualBalance().

```
function deposit(uint256 amount, bytes32 reason) external nonReentrant {
    uint256 beforeBal = hackToken.balanceOf(address(this));
    hackToken.safeTransferFrom(msg.sender, address(this), amount);
    uint256 received = hackToken.balanceOf(address(this)) - beforeBal;
    require(received == amount, "fee-on-transfer not supported");
    poolBalance += received;
    totalReceived += received;
    emit Deposited(msg.sender, received, reason);
}
```

Evidencia: HC-M01, fuzz Foundry 256 runs y contraejemplo Halmos.

8.2 M-02 - Exención de comisión inmediata y elegibilidad obsoleta

La documentación exige 100.000 HACK en staking **durante** 12 meses. La implementación habilita el beneficio al comprometerse a una duración anual, sin esperar el aniversario. Además, el booleano puede permanecer activo después de retirar la posición anual si otra posición mantiene el agregado por encima de 100.000.

Reproducción 1: stake anual -> activateNoCommission() inmediatamente.
Reproducción 2: stake anual + stake mensual -> activar -> vencer/retirar anual -> el estado continúa activo por el agregado mensual.

La elegibilidad debe derivarse de una posición concreta, no de un booleano persistente ni del total agregado.

```
function hasNoCommission(address user) public view returns (bool) {
    uint256 id = qualifyingStakeId[user];
    Stake memory s = stakes[user][id];
    return s.active
        && s.amount >= 100_000 ether
        && block.timestamp >= s.startTime + 365 days;
}
```

Pruebas: HC-H03 y BL-P04.

8.3 M-03 - Reclamación de reputación falla al cruzar la época

registerWinner() guarda bajo una época; claimBonus() busca la época calculada al reclamar. Un ganador registrado al final de una época pierde una ventana de tres días aún vigente al cruzar el límite.

Solución: utilizar bonusId o cycleId explícito y validar registeredAt + 3 days.

```
function claimBonus(bytes32 bonusId) external {
    Bonus storage b = bonuses[bonusId];
    require(b.winner == msg.sender, "not winner");
    require(block.timestamp <= b.registeredAt + 3 days, "window closed");
    require(!b.claimed, "claimed");
    b.claimed = true;
    pool.distribute(msg.sender, b.amount, "reputation");
}
```

Prueba: `HC-M02`.

8.4 M-04 - Eventos off-chain sin IDs únicos ni idempotencia

Asistencias, contrataciones, eventos y algunos bonos son contadores o pagos directos sin un identificador consumible. Un retry del backend o una firma repetida puede contar el mismo hecho varias veces.

Flujo de reproducción: llamar dos veces `rewardTalentHired(educator, talent)` para la misma contratación; ambas llamadas pagan porque no existe `hiringId`.

```
mapping(bytes32 => bool) public processedAction;

function rewardHiring(bytes32 hiringId, address educator, address talent)
    external onlyRole(ENFORCER_ROLE)
{
    require(!processedAction[hiringId], "replay");
    processedAction[hiringId] = true;
    pool.distribute(educator, 5_000 ether, "educator_hiring_bonus");
}
```

Prueba: `HC-M04`.

8.5 M-05 - `Ownable` y `AccessControl` se transfieren de manera divergente

Transferir `owner()` no transfiere `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `BURNER_ROLE` o `PAUSER_ROLE`. El operador anterior conserva capacidades críticas y el nuevo owner puede no administrar nada.

Solución: usar un solo modelo, preferentemente `AccessControlDefaultAdminRules`, administrador multisig y transferencia en dos pasos. Verificar que la cuenta anterior renuncie después de que la nueva acepte.

```
contract HackToken is ERC20, AccessControlDefaultAdminRules {
    constructor(address admin)
        ERC20("HackChain", "HACK")
        AccessControlDefaultAdminRules(2 days, admin)
    {}
}
```

Prueba: `HC-L01`.

8.6 M-06 - Bonos de recruiter no prueban actividad, registro ni KYC coherente

“Activo durante 7 días” se implementa como tiempo transcurrido desde el registro; no existe actividad. El KYC puede pagarse sin exigir registro y las contrataciones carecen de ID.

Solución: una attestation firmada debe incluir recruiter, proveedor KYC, resultado, identificador único, fecha de emisión, vencimiento y nonce. El contrato debe exigir `isRegistered` y consumir el ID una vez.

```
function claimKycBonus(KycAttestation calldata a, bytes calldata sig) external {
    require(isRegistered[a.recruiter], "not registered");
    require(a.recruiter == msg.sender, "wrong claimant");
    require(!usedKyc[a.kycId], "kyc used");
    require(_verifyKycSigner(a, sig), "invalid attestation");
```

```
    usedKyc[a.kycId] = true;
    pool.distribute(msg.sender, 200_000 ether, "kyc_bonus");
}
```

9 Hallazgos Low e Informational

9.1 L-01 - Transferencias ERC-20 no seguras

Varias llamadas `transfer/transferFrom` usan el booleano manualmente o lo ignoran. Aplicar `SafeERC20` de manera uniforme y validar que las dependencias sean contratos.

9.2 L-02 - Índice de stake validado tarde

La referencia al array se obtiene antes de comprobar `stakeIndex < length`, generando un panic en lugar del custom error. Validar primero y acceder después.

9.3 L-03 - Membresía avanzada vencida conserva estado activo

`active` puede seguir en `true` tras el vencimiento. Esto bloquea reactivaciones y permite cobrar penalización por cancelar una membresía ya vencida. Derivar actividad de `expiresAt` o sincronizar el estado antes de cada acción.

9.4 L-04 - Bucles y lotes no acotados

La búsqueda de stakes y la distribución a arrays de talentos pueden superar el gas del bloque. Mantener índices agregados, limitar el batch y usar distribución pull/paginada.

9.5 L-05 - Patrocinador sobrescribible y sin reembolso

Una segunda financiación reemplaza `projectSponsor[projectId]`, aunque acumula ambos montos. No hay deadline ni cancelación. Modelar contribuciones por sponsor y permitir refund del remanente al vencer.

9.6 I-01 - Épocas de 30 días no equivalen a meses calendario

Los mecanismos documentados como “mes” usan `block.timestamp / 30 days`. Definir formalmente `cycleId`, inicio, fin, zona horaria de la capa off-chain y periodo de reclamación.

9.7 I-02 - Recompensas no reservadas

Ser elegible no reserva fondos. Otro módulo puede agotar el pool antes del claim. Crear presupuestos y reservas por módulo/ciclo respaldadas por balance real.

9.8 I-03 - Despliegue y roles no reproducibles desde el repositorio recibido

Faltan scripts idempotentes, addresses, CI y grants/revokes. El paquete de auditoría incluye un entorno de reproducción, pero el proyecto debe incorporar su propio pipeline productivo.

9.9 I-04 - Calidad y observabilidad

`ReputationBunises.sol` contiene un typo, hay estado/eventos sin uso y setters críticos sin eventos. Corregir nomenclatura, eliminar estado muerto y emitir eventos para cambios de dependencias y roles.

9.10 L-06 - La pausa de emergencia no es granular y puede bloquear retiros

Observación. HackToken dispone de una pausa global en `_update()`. En la práctica, esa pausa evita nuevos `transferFrom` hacia staking, membresías o el pool, pero también impide que `StakingContract` devuelva el principal mediante `transfer`. Los módulos que reciben fondos no heredan `Pausable` ni ofrecen una modalidad explícita de "solo retiros".

Riesgo. Ante un incidente en staking o membresías, el equipo solo puede dejar el sistema operando o pausar el token completo. La segunda opción reduce la superficie de ataque, pero también congela salidas legítimas y agrava H-02.

Recomendación. Implementar pausas por módulo y por dirección de flujo. Las operaciones que crean nueva exposición (`stake`, activación/renovación de membresía, nuevas financiaciones y depósitos administrativos) deben usar `whenNotPaused`. Los retiros de principal, cancelaciones seguras, pagos de deuda y funciones de rescate deben permanecer disponibles siempre que el token no esté comprometido. El rol de emergencia debe estar en multisig, con eventos, runbook y pruebas de pausa/despausa.

```
contract StakingContract is AccessControl, Pausable, ReentrancyGuardTransient {
    bytes32 public constant EMERGENCY_ROLE = keccak256("EMERGENCY_ROLE");

    function pauseDeposits() external onlyRole(EMERGENCY_ROLE) {
        _pause();
    }

    function unpauseDeposits() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _unpause();
    }

    function stake(uint256 amount, uint256 duration)
        external
        whenNotPaused
        nonReentrant
    {
        _stake(msg.sender, amount, duration);
    }

    // Deliberadamente sin whenNotPaused: permite recuperar el principal.
    function unstake(uint256 stakeId) external nonReentrant {
        _withdrawPrincipal(msg.sender, stakeId);
    }
}
```

Criterio de aceptación. Con depósitos pausados, `stake()` debe revertir y `unstake()` debe seguir devolviendo el principal. La pausa global del token debe reservarse para incidentes del propio ERC-20.

9.11 I-05 - Mappings públicos y getters duplicados

Se confirmaron al menos **16 accesos escalares exactamente duplicados**, además de wrappers de structs. Ejemplos: `apiIntegrationRewarded/hasApiIntegrationBonus`, `legacyCertsCount/getLegacyCertsCount`, `firstEventRewarded/hasFirstEventReward`, `profileBlocked/isProfileBlocked`, `isRegistered/getIsRegistered`, `referredBy/getReferrer`, `totalStakedByUser/getTotalStaked` y `degreeRewarded/hasDegreeReward`.

Esto no crea una explotación directa, pero aumenta el bytecode de despliegue, duplica entradas en ABI, amplía la superficie que frontend/indexadores deben mantener y expone la forma del almacenamiento como API pública.

Recomendación profesional. Elegir una sola estrategia:

- Para datos simples que realmente forman parte de la API estable, conservar el mapping `public` y eliminar el wrapper exacto.

- Para estados derivados, structs, arrays o datos que pueden cambiar de representación, usar mappings `private` con prefijo _ y getters explícitos.
- Para arrays crecientes, evitar devolver la colección completa; exponer conteo, lectura por índice y paginación.

```
mapping(address user => uint256 amount) private _totalStaked;

/// @notice Returns the active principal currently staked by `user`.
/// @param user Account whose active principal is queried.
/// @return amount Amount of HACK currently staked.
function totalStaked(address user) external view returns (uint256 amount) {
    return _totalStaked[user];
}
```

La eliminación o renombre de getters es un cambio de ABI y debe coordinarse con frontend, indexadores y pruebas.

9.12 I-06 - NatSpec incompleto y parcialmente desalineado

El repositorio sí contiene documentación NatSpec en la mayoría de funciones, por lo que no parte de cero. Sin embargo, la revisión heurística de las declaraciones públicas/externas encontró interfaces mínimas sin documentación, numerosos parámetros sin `@param` y **ninguna función con retorno documentado mediante `@return`**. Los errores personalizados tampoco explican contexto o valores, y varias variables públicas solo tienen comentarios ordinarios.

También existen afirmaciones que no coinciden exactamente con el código. Por ejemplo, el NatSpec de la exención de comisión dice “staked for at least 12 months”, mientras la función solo comprueba una duración elegida de un año; además, comentarios heredados como `CHANGE 3`, `CHANGE 4`, etc. describen historial de edición y no la especificación final.

Recomendación. Documentar toda la ABI con `@notice`, `@dev`, `@param` y `@return`; describir roles, trust assumptions, efectos de pausa, orden CEI, unidades, ciclos, eventos y errores; y añadir invariantes y motivos de seguridad donde el comportamiento no sea obvio. El NatSpec debe describir lo que el código hace ahora, no lo que se planeó o cambió durante el desarrollo.

```
/// @notice Withdraws a matured staking principal and records its reward as debt.
/// @dev Uses checks-effects-interactions.
///      Remains available while deposits are paused.
/// @param stakeId Index of the caller's staking position.
/// @return principal Amount of HACK returned to the caller.
/// @return rewardDebt Reward recorded for later collection from the incentives pool.
function unstake(uint256 stakeId)
    external
    nonReentrant
    returns (uint256 principal, uint256 rewardDebt)
{
    // ...
}
```

9.13 I-07 - Modernización condicional a `ReentrancyGuardTransient`

El commit auditado usa OpenZeppelin 5.0.2 y `ReentrancyGuard`. En la documentación actual de OpenZeppelin 5.x, el guard basado en almacenamiento está marcado como deprecado para su reemplazo en v6 y se recomienda considerar `ReentrancyGuardTransient` cuando EIP-1153 esté disponible. La variante transient existe desde OpenZeppelin 5.1 y solo funciona en redes compatibles.

Polygon PoS incluyó EIP-1153 en PIP-31. Se realizó una compilación de compatibilidad separada con OpenZeppelin 5.6.1, Solidity 0.8.24 y `evmVersion=cancun`; los contratos compilaron después de sustituir importación e herencia. Esta prueba no reemplaza pruebas de integración en la red objetivo.

Dictamen. La migración aplica como modernización y ahorro de gas, no como corrección de una vulnerabilidad demostrada: el `ReentrancyGuard` actual sigue cumpliendo su función. Debe actualizarse la

dependencia de manera controlada, fijar versión exacta, activar EVM Cancun, ejecutar toda la suite, medir bytecode/gas y probar en fork/testnet de la cadena concreta.

```
import {ReentrancyGuardTransient} from
    "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";

contract MembershipSystem is AccessControl, ReentrancyGuardTransient {
    // Los modificadores nonReentrant existentes se conservan.
}
```

No debe desplegarse bytecode con TLOAD/TSTORE en una red que no soporte EIP-1153. Para despliegues multichain, mantener perfiles de compilación o artefactos separados.

9.14 Criterio revisado sin hallazgo - Convención `Contract__Error()`

La convención `Contract__Error()` es válida, pero no es un estándar obligatorio de Solidity ni de OpenZeppelin. Los errores declarados dentro de un contrato ya están contextualizados por su ABI y pueden referenciarse en pruebas como `StakingContract.InvalidDuration.selector`. OpenZeppelin utiliza nombres semánticos como `ERC20InsufficientBalance` o `AccessControlUnauthorizedAccount`, pero no exige el patrón de doble guion bajo.

Por ello, no se recomienda renombrar todos los errores únicamente para adoptar esa estética: generaría un cambio de ABI y obligaría a actualizar tests, frontend e indexadores sin mejorar la seguridad. Sí se recomienda mejorar la semántica y añadir parámetros útiles:

```
error InvalidAddress(address account);
error InsufficientPoolBalance(uint256 available, uint256 required);
error StakeNotMatured(uint256 unlockAt, uint256 currentTime);
```

Un prefijo de contrato puede adoptarse como guía interna cuando los errores se declaren a nivel de archivo, se compartan entre módulos o se mezclen ABIs, siempre que el proyecto sea consistente.

10 Brechas de conformidad de negocio

10.1 BL-01 - Distribución, vesting y bloqueo de liquidez no se fuerzan on-chain (High)

Documentación. Oferta máxima de 1.000.000.000 HACK; categorías 10/5/10/15/30/20/10%; asesores reciben 1% desde tesorería; vesting del equipo, asesores, preventa y venta privada; liquidez inicial bloqueada al menos un año.

Observación. HackToken limita la oferta máxima, pero un `MINTER_ROLE` puede mintear hasta el cap a cualquier dirección. No existen vaults por categoría, calendarios de vesting o contrato de LP lock en el alcance.

Riesgo. Un error o compromiso del minter puede romper públicamente la tokenomics sin violar el cap global.

Solución. Mintear las asignaciones iniciales directamente a vaults independientes (`VestingWallet` o implementación de vesting por tranche), revocar el minter tras completar el plan o limitarlo por categoría. Bloquear los LP tokens/NFT de la posición de liquidez mediante un locker verificable.

```
VestingWallet teamVault = new VestingWallet(
    teamMultisig,
    uint64(tge + 365 days),
    uint64(730 days)
);
hack.mint(address(teamVault), 100_000_000 ether);
```

Debe definirse si “12 meses de bloqueo + 24 meses de liberación” significa cliff de 12 meses y vesting lineal durante los 24 siguientes. La implementación debe reflejar esa decisión en pruebas de frontera.

10.2 BL-02 - Comisión del 5% por transacción ausente (High)

El plan y los mecanismos establecen que ciertas transacciones de plataforma pagan 5% en HACK a tesorería, salvo usuarios elegibles. No existe `CommissionSystem` ni contrato que consulte `hasNoCommission`.

Riesgo. El flujo principal de ingresos de tesorería no está implementado y la exención de staking no tiene efecto económico verificable.

Solución. Centralizar settlement en un router: calcular fee, cobrar HACK, enviar neto/destino y consultar elegibilidad en el mismo estado.

```
function settle(
    address payer,
    address payee,
    uint256 gross,
    bytes32 actionId
) external {
    require(!processed[actionId], "duplicate");
    processed[actionId] = true;

    uint256 fee = staking.hasNoCommission(payer) ? 0 : gross * 5 / 100;
    hack.safeTransferFrom(payer, treasury, fee);
    hack.safeTransferFrom(payer, payee, gross - fee);
    emit Settled(actionId, payer, payee, gross, fee);
}
```

Se debe decidir si la comisión se suma al precio (`gross + fee`) o se descuenta del receptor (`gross - fee`).

10.3 BL-03 - Clases personalizadas no implementadas (Medium)

Falta el flujo donde el educador fija un precio mínimo de 1.000 HACK por hora, el talento agenda, asiste y el educador recibe el pago.

Solución. Crear `LessonEscrow`: oferta con precio/duración, aceptación del talento, depósito, confirmación bilateral, timeout, cancelación y disputa. No pagar directamente al reservar.

```
function book(bytes32 lessonId, address educator, uint256 amount) external {
    require(amount >= 1_000 ether, "minimum");
    hack.safeTransferFrom(msg.sender, address(this), amount);
    lessons[lessonId] = Lesson(msg.sender, educator, amount, Status.Funded);
}
```

10.4 BL-04 - Membresía avanzada incompleta (Medium)

Solo se implementa el nivel mensual de 50.000 HACK. Faltan trimestral 140.000 y anual 550.000. Cancelar cobra 1.000 HACK adicionales, pero no devuelve la porción no consumida, aunque el diagrama muestra “solicitar reembolso”.

Solución. Definir tiers, prorrateo/refund y tratamiento contable de fondos ya enviados a tesorería/pool. La alternativa más segura es mantener el pago en escrow y liberar por periodo; si la política real es “sin reembolso”, eliminar esa ruta de todos los documentos.

10.5 BL-05 - Cancelación de membresía académica no implementada y fuentes contradictorias (Medium)

La tabla de mecanismos describe precios/reparto, mientras el diagrama de plan de negocio muestra cancelación, penalización y solicitud de reembolso. El contrato no contiene cancelación académica.

Acción requerida. Aprobar una especificación única antes de codificar. No implementar a partir de una inferencia del auditor.

10.6 BL-06 - Financiación OSS no valida roles, repositorio ni talentos (Medium)

`fundProject` acepta cualquier wallet, un `projectId` cero y múltiples aportes que sobrescriben sponsor. La verificación de GitHub, open source y talentos activos solo aparece en comentario.

Solución. Crear un registro de proyectos con attestation del repo/commit, roles permitidos, lista/Merkle root de talentos, deadline y refund. Los payouts deben ser pull-based.

10.7 BL-07 - Suplantación y plagio no siguen el flujo documentado (Medium)

La penalización por suplantación llama `_applyPenalty(..., blockProfile=false, ...)`, por lo que no bloquea el perfil. Los documentos alternan entre eliminación del perfil y bloqueo hasta corregir. Para plagio externo, los tokens llegan a tesorería, pero no existe operación atómica/obligatoria de pago al afectado. `unblockProfile` no referencia `caseId`, corrección o pago.

Solución. Modelar casos explícitos con estados `Reported -> Confirmed -> DebtCreated -> Corrected/Paid -> Closed`, evidencia hash y permisos separados para confirmar/cerrar.

10.8 BL-08 - Hechos de negocio dependen de confianza off-chain sin attestations (Medium)

Asistencia, 15 minutos de entrevista, contratación, grado, actividad, KYC, API, plagio y empleo son datos externos. En el diseño actual basta la llamada de un enforcer, generalmente sin ID ni firma del proveedor.

Solución. EIP-712 + nonces + expiración + `actionId`. Para hechos complejos, usar un registry de attestations y hacer que los módulos de recompensas consuman esa evidencia una vez.

10.9 BL-09 - DAO/gobernanza documentada pero fuera del repositorio (Informational)

El whitepaper describe participación de la comunidad y la presentación afirma que el staking permite participar en DAO. No hay contrato de gobernanza en el alcance. Como el roadmap lo presenta como una implementación futura, se clasifica como brecha de alcance, no como vulnerabilidad del código recibido.

Antes de lanzarla se requiere definir snapshot de votos, delegación, quorum, timelock, prevención de flash-loan voting y qué parámetros pueden modificarse.

10.10 BL-10 - Contradicciones documentales y exposición regulatoria (Medium)

Se encontraron, entre otras, las siguientes inconsistencias:

- Venta masiva: una tabla indica 5% **de la venta**; el diagrama muestra 5% **del total de tokens**.
- Suplantación: una fuente habla de eliminar perfil; el diagrama habla de bloquear, corregir y desbloquear.
- Membresías: los diagramas muestran reembolso/cancelación que las tablas o contratos no definen completamente.
- Plan de negocio: se prometen porcentajes de tesorería y rendimientos mensuales a inversionistas durante tres años, mientras el descargo afirma que no hay derechos económicos ni expectativas de ganancias.

Riesgo. El equipo de ingeniería no puede implementar una regla correcta cuando la fuente normativa es ambigua. Además, un descargo no necesariamente neutraliza la sustancia económica descrita en otras páginas.

Acción requerida. Crear una especificación versionada y aprobada por producto, tokenomics, seguridad y asesoría legal. Cada mecanismo debe tener fórmula, actor, fuente/destino, momento de snapshot, estados, cancelación, replay protection y política de disputa.

11 Arquitectura de remediación recomendada

11.1 Capa 1 - Token y tesorería

- HackToken con administración multisig y `AccessControlDefaultAdminRules`.
- Vaults de vesting por asignación.
- Minter limitado/revocado después de TGE.
- Treasury y pool separados, con presupuestos por módulo.
- LP lock verificable.

11.2 Capa 2 - Evidencia de negocio

- `AttestationRegistry` EIP-712.
- IDs únicos para evento, entrevista, contratación, KYC, certificado, vista y proyecto.
- Nonces, expiración, versión de schema y revocación.
- Roles de emisores separados por tipo de evidencia.

11.3 Capa 3 - Pagos y recompensas

- `CommissionRouter` para settlement y comisión.
- `RewardBudget` con reservas por mecanismo/ciclo.
- `LessonEscrow` y `ProjectEscrow`.
- Staking con principal desacoplado de reward.
- Reputación con ciclos explícitos y snapshots.

11.4 Capa 4 - Penalizaciones

- Sistema de casos y deuda, no `transferFrom` coercitivo improvisado.
- Snapshot/attestation del hecho y base económica.
- Bloqueo de perfil independiente del cobro.
- Disputa, corrección, pago, cierre y auditoría de cada caso.

12 Plan de corrección priorizado

12.1 Bloqueante antes de testnet pública

1. Corregir H-01 a H-05.
2. Implementar BL-01 y BL-02 o eliminar sus promesas del lanzamiento.
3. Definir formalmente la especificación de penalizaciones y membresías.
4. Incorporar IDs/attestations para acciones remuneradas.
5. Crear scripts reproducibles de despliegue y matriz de roles.
6. Añadir pausa granular de entradas sin bloquear retiros de principal.

12.2 Antes de mainnet

1. Resolver todos los Medium y Low.
2. Añadir presupuestos/reservas y monitoreo de solvencia.
3. Revisar legalmente ventas privadas, retornos y documentación pública.
4. Ejecutar pruebas de integración con backend y servicios de identidad.
5. Re-auditar el diff final y realizar simulación de despliegue en fork/testnet.
6. Eliminar getters duplicados, completar NatSpec y decidir formalmente el perfil `ReentrancyGuard`/`ReentrancyGuardTransient` por red.

13 Pruebas de aceptación mínimas después de corregir

- Referido no cobra antes de 30 días y cada `referralId` solo se consume una vez.

- Staker recupera principal con pool vacío, token pausado y rol revocado.
- Auto-vistas y vista duplicada revierten; snapshots no cambian después del cierre.
- Una sanción crea bloqueo/deuda aunque el usuario tenga allowance cero o mueva el saldo.
- Venta masiva usa el balance anterior a la venta y supply obtenido de una fuente definida.
- `poolBalance <= actualBalance` para toda secuencia de depósitos/distribuciones.
- Exención solo existe tras 365 días de una posición activa concreta.
- Claims de reputación funcionan al cruzar fin de ciclo dentro de la ventana.
- Todos los eventos/hirings/KYC duplicados revierten.
- Transferir administración elimina permisos de la cuenta anterior.
- Suma de asignaciones y vesting coincide exactamente con 1.000.000.000 HACK.
- Comisión del 5% se aplica o exime correctamente en el settlement real.
- Cancelación/reembolso de membresía coincide con la especificación aprobada.
- Al pausar depósitos, nuevas entradas revierten y los retiros de principal siguen funcionando.
- La build cancun con `ReentrancyGuardTransient` se prueba en fork/testnet EIP-1153; una build no compatible nunca se despliega en cadenas antiguas.
- La ABI no contiene getters escalares duplicados y toda función pública/externa tiene NatSpec completo y correcto.

14 Guía de reproducción del paquete

```
unzip Hackchain_Smart_Contract_Audit_2026-07-11.zip
cd Hackchain_Smart_Contract_Audit_2026-07-11
npm ci
npm run compile
npm run test:js
npm run test:forge
npm run lint:forge
```

Archivos relevantes:

- `test/security.spec.js`: 16 PoC locales.
- `foundry/test/SecurityPoC.t.sol`: pruebas Foundry.
- `foundry/test/HalmosPoolInvariant.t.sol`: propiedad simbólica.
- `evidence/mocha-security.txt`: salida de las 16 PoC.
- `evidence/slither.*`: ejecución y JSON de Slither.
- `evidence/halmos.*`: contraejemplo simbólico.
- `reports/contracts/`: revisión archivo por archivo.
- `agent-handoff/`: checklist y pendientes para el agente técnico.

15 Limitaciones

Una auditoría reduce riesgo, no demuestra ausencia absoluta de vulnerabilidades. Los resultados aplican al commit y dependencias indicados. Cualquier cambio de lógica, rol, dirección, backend, tokenomics o contrato requiere análisis del diff. Los mecanismos que dependen de hechos del mundo real solo son tan seguros como el proveedor de evidencia, sus claves y su proceso de disputa.

16 Conclusión final

El código demuestra una base modular útil, pero actualmente la seguridad no coincide con el nivel de confianza prometido por el modelo de negocio. La prioridad no es agregar más bonos: es asegurar la custodia del principal, hacer verificables las condiciones, cerrar replays, definir snapshots y garantizar que los flujos económicos centrales realmente existen.

No se recomienda desplegar el sistema auditado con fondos reales. La pausa granular, la limpieza de ABI, el NatSpec y la actualización del guard mejoran resiliencia y mantenibilidad, pero no sustituyen la corrección de los High/Medium. Después de corregir High/Medium y resolver la especificación, debe realizarse

una re-auditoría enfocada en el diff, scripts de despliegue, roles, tokenomics y la integración backend-attestations.

17 Apéndice A - Evidencia y entorno técnico

Elemento	Resultado
Commit	`290377e9862aeabaa2814bcd0bac9ddad2707ce8`
Solidity	`0.8.24`
OpenZeppelin del commit auditado	`5.0.2`
OpenZeppelin para prueba de modernización	`5.6.1`
ReentrancyGuardTransient	Compilación compatible con solc 0.8.24 + `evmVersion=cancun`; pendiente prueba en red objetivo
Mocha/Ganache PoC	16 passing
Foundry	7 passing; fuzz 256 runs
Slither	Ejecución exitosa; señales triadas manualmente
Halmos	Contraejemplo esperado para solvencia de `deposit()`
Mythril	Instalación no finalizada en el entorno; documentado en handoff
Echidna	No disponible; propiedades propuestas en handoff

18 Apéndice B - Hashes de documentos de negocio

Mecanismos del token HACK.pdf

`6cfbaf0982ed551b2e5b82cc49542a4d7cee0dba9ac3d7a4703dbf1b510ee124`

Plan de negocios.pdf

`5004babd0b771e28ba125200f8b156077426a15adfe8d62def4ab1d4c31ad856`

Whitepaper HackChain 1.1.0 (1).pdf

`1d08f647bbefd217a2c5bc190666d2b5876c1e8e69a17ef07a11aa704b5a8fba`

Diagramas de mecanismos.pdf

`bd63a573e4c9d3c345fbc39c162bf9107ba7ab845b505f8992bf33022bd8c4d0`

Conoce Hack Chain_20260508_183540_0000_compressed.pdf

`d341a58235f336d890dbec88fd15bd87952b414b21ab05abe9ca89982696b257`

19 Apéndice C - Referencias técnicas

- Solidity Style Guide: convenciones de nombres y prioridad de consistencia del proyecto.
- Solidity NatSpec Format: recomendación de documentar completamente toda la interfaz pública/ABI.
- OpenZeppelin Contracts 5.x Utilities: estado de `ReentrancyGuard` y condiciones de `ReentrancyGuardTransient`.
- OpenZeppelin Contracts v5.6.1: versión estable consultada para la prueba de modernización.
- Polygon PIP-31: inclusión de EIP-1153 en el conjunto Cancun de Polygon PoS.
- Evidencia local: `evidence/modernization/oz-5.6.1-reentrancyguardtransient-compile.txt`.

20 Apéndice D - Nota sobre severidad

- **Critical:** pérdida/captura sistémica inmediata o control total sin condiciones realistas.

- **High:** pérdida relevante, bloqueo de fondos o quiebre central del modelo con condiciones alcanzables.
- **Medium:** incumplimiento explotable o riesgo material que depende de actor privilegiado, timing o integración.
- **Low:** robustez, gas, compatibilidad o lógica secundaria con impacto limitado.
- **Informational:** deuda técnica, alcance futuro o recomendaciones de operación.